Handwritten Equation Solver

CS351 Final Report - Tobi Adegbite

Project Supervisor: Weiren Yu

Abstract

The aim of this project is to develop a web-application that allows users to get near-instant answers to handwritten mathematical equations.A Watch, Attend, Parse (WAP) baseline model was initially developed, followed by a Counting-Aware Network (CAN) which builds on the baseline WAP model. Furthermore, a MathEngine powered by SymPy was developed to route and solve parsed algebraic, calculus, and basic arithmetic expressions.

Keywords: Optimisation, Machine Learning, Recurrent Neural Network, Convolutional Neural Network, Autoregressive Decoder, Deep Learning, Algorithms, Gradient Descent.

Contents

1 Introduction

Recognizing handwritten mathematical equations poses unique challenges compared to cursive or printed text due to the two-dimensional spatial structure of mathematical notation. An application with this capability would have numerous applications across various contexts such as education, office automation, e.t.c[34].

This application will provide the solution to a mathematical equation in a given image. The end goal is to make it easy for anyone using this , regardless of mathematical knowledge, to get quick answers to provided equations. Normally, equations are solved manually. However, in more advanced topics, such which include specific types of integrals, the process can become highly time-consuming, and errors in calculation are very likely. This project aims to solve this problem by allowing users to upload images of handwritten equations, and the application will provide an answer to the provided equation, hence skipping the need for a user to solve it themselves. The final solver aims to have the ability to solve integrals with limits, derivatives , basic arithmetic and algebraic equations.

1.1 Background

1.1.1 Predefined-Grammar Based HMER's

Earlier handwritten recognition systems used a set of hard coded production rules to reduce handwritten equations into one single math object. The most rigorous implementation of this was through Stochastic Context-Free Grammars (SCFG)[5].

$S \rightarrow \langle Expression \rangle$
$\langle Expression \rangle \rightarrow \langle Factor \rangle : \langle Expression \rangle$
$\langle Expression \rangle \rightarrow \langle Factor \rangle$
$\langle Factor \rangle \rightarrow \langle ExprOverExpr \rangle$
$\langle Factor \rangle \rightarrow \langle Symbol \rangle$
$\langle Factor \rangle \rightarrow \langle ExprBigOpExpr \rangle$
$\langle Factor \rangle \rightarrow \langle SymbSupSub \rangle$
$\langle ExprOverExpr \rangle \rightarrow \langle Expression \rangle / \langle OverExpr \rangle$
$\langle OverExpr \rangle \rightarrow \langle Over \rangle / \langle Expression \rangle$
$\langle ExprBigOpExpr \rangle \rightarrow \langle Expression \rangle / \langle BigOpExpr \rangle$
$\langle BigOpExpr \rangle \rightarrow \langle BigOp \rangle / \langle Expression \rangle$
$\langle SymbSupSub \rangle \rightarrow \langle Symbol \rangle : \langle SupScript \rangle$
$\langle SymbSupSub \rangle \rightarrow \langle Symbol \rangle : \langle SubScript \rangle$
$\langle SymbSupSub \rangle \rightarrow \langle Symbol \rangle : \langle SupSubScript \rangle$
$\langle SupScript \rangle \rightarrow \langle Expression \rangle$
$\langle SubScript \rangle \rightarrow \langle Expression \rangle$
$\langle SupSubScript \rangle \rightarrow \langle Expression \rangle / \langle Expression \rangle.$

Figure 1: Grammar Production rules

The grammar for the equation recognition task, on a conceptual level, had the following production rules in figure 1. Here the start symbol , S, represents an $\langle Expression \rangle$ i.e $\frac{\pi^2}{6} = \sum_{n=1}^{\infty} \frac{1}{n^2}$, which in turn could could be represented as a $\langle Factor \rangle$ followed horizontally by an $\langle Expression \rangle$. A $\langle Factor \rangle$ is a nonterminal which represents an expression that cannot be horizontally subdivided , for example $\frac{\pi^2}{6}$[5]. On a conceptual level these rules work perfectly but on a spatial level i.e when implementing the system in practice, the system will have to use **hard-coded** geometric constraints to determine if symbols satisfy these rules. For example, measuring the gap between regions to ensure they are close enough to be considered part of the same $\langle Expression \rangle$. Another example would be distinguishing between a $\langle Symbol \rangle$ being a 2 or z, if the handwritten term is slightly misaligned by even 0.2%, the wrong result could be inferred. This means that with this approach, both ambiguity and ordering are critical issues which, if not handled effectively, will lead to algorithms which are expensive and will most likely yield poor results[10]. To fully grasp why having grammar production rules is not a very feasible idea, we will look at the best adaptation of this concept which is known as the 2D Cocke-Younger-Kasami (CYK) Algorithm.

Before we get to this, we need to understand the fundamental mechanism of the CYK (Cocke-Younger-Kasami) Algorithm. The CYK algorithm was introduced in the 1960s[21, 35]. and is one of the classic examples of dynamic programming in formal linguistics. Its basic function is membership testing,

i.e., whether a particular string of symbols (an input) can be generated by a given set of grammatical rules (a Context-Free Grammar). CYK is a bottom-up parser[7]. Unlike top-down parsers which start with a general concept (like a sentence) and try to figure out what words fit into it, CYK starts with raw data (words or symbols) and goes up. Top-down parsers will have to guess what rule to try next, when building the string and if guessed wrong, backtrack to the source of the mistake but the CYK algorithm will combine smaller recognized units into larger, more complex structures until it knows if the whole input is valid in that language (grammar rules). Note that the grammar being used must be in Chomsky Normal Form (CNF). Chomsky Normal Form (CNF) is a special way of writing a context-free grammar where every rule follows a strict pattern. [4]

Non-terminal (Variable)	**Production Rules**
S	AB | BC
A	BA | a
B	CC | b
C	AB | a

Table 1: CNF Grammar Example

An example of a valid CNF grammar that can be used in the standard CYK algorithm is shown in table 1. Each capital letter refers to a variable or non-terminal while each smaller letter refers to a terminal.

Algorithm 1: Standard CYK Parsing Algorithm. Adapted from the formal matrix definitions by Younger [35] and Hopcroft *et al.*

Input: A string of symbols $w = w_1 w_2 ... w_n$ and a CFG G in CNF

Output: A triangular table T where $T[i, j]$ contains the symbols that can derive $w_i ... w_j$

```
// Step 1:  Fill the base of the table (Terminals)
for i = 1 to n do
    for each rule A → w_i do
        Add A to T[i, 1]
    end
end

// Step 2:  Build the tree upwards (Non-terminals)
for j = 2 to n do
    // Length of the span
    for i = 1 to n − j + 1 do
        // Start position
        for k = 1 to j − 1 do
            // Split point
            if B ∈ T[i, k] and C ∈ T[i + k, j − k] then
                for each rule A → BC in G do
                    Add A to T[i, j]
                end
            end
        end
    end
end
return T[1, n] contains the Start symbol S
```

Before moving on, it will be beneficial to note that the table this algorithm yields, will be one where i reflects columns and j reflects rows. To understand the CYK parsing process, we need to examine the mechanics of the algorithm line by line. The algorithm, shown in, 1 is implemented in two stages namely, terminal initialization and dynamic programming.

PHASE 1: TERMINAL INITIALIZATION (LINES 1-5):

The algorithm starts by iterating through the input string w of length n. For

each character w_i, it considers the grammar for any terminal production rule $A \rightarrow w_i$. If there is a match, the non-terminal A is placed at the bottom of the parsing table $T[i, 1]$. The algorithm is, in mathematical terms, starting from the base case of the dynamic programming matrix and treating each character as a substring of length 1. This phase takes $O(n)$ time to complete as it only takes one pass through the string and only one rule lookup.

Phase 2: The Recursive Step (Lines 6-16):

The main logic of the algorithm lies here, the triple nested loops that are responsible for its $O(n^3)$ time complexity.

- **The Outer Loop (Line 8, j):** This loop controls the length of the substring now being evaluated and this is increased from length 2 to the full length of the string n. By enforcing this bottom-up approach, the algorithm guarantees that any smaller substrings required for a calculation have been already evaluated.
- **The Middle Loop (Line 9, i):** Next, this loop determines the starting index of the substring. As the length j of a substring increases, the number of valid starting positions is naturally reduced (e.g., a substring of length 5 can only start at index 1 in a 5-letter word, so $n - j + 1$).
- **The Inner Loop (Line 10, k):** This is a split point mechanism in order to check if a substring from index i with length j is valid. The algorithm tries to split it into two smaller substrings. It checks if the left portion $T[i, k]$ and the right portion $T[i + k, j - k]$ can be combined using a binary rule $A \rightarrow BC$ (Lines 11-13).

1.1.1.1 2D Cocke-Younger-Kasami (CYK) Algorithm.

The transition from 1D to 2D requires an important change in the CYK algorithm. 1D text has a strictly left-to-right order of characters that the algorithm then only looks for contiguous substrings. In 2D space one stroke could be combined with any stroke on the page. The standard way to do this in HMER is the one defined by Alvaro et al. [1] that looks for subsets of strokes instead of linear spans and tests every possible partition against a set of spatial relations R (Horizontal, Superscript, Subscript, Above, Below)

Algorithm 2: 2D-CYK Parsing Algorithm for Stochastic Context-Free Grammars. Adapted and reformulated from the spatial subset partitioning logic defined by Alvaro *et al.* [1].

Input: A set of observed symbols $O = \{s_1, s_2, ..., s_n\}$, 2D-SCFG Grammar G, Set of Spatial Relations R

Output: The most probable parse tree $T[O]$ for the entire mathematical expression

```
// Phase 1: Initialization (Terminals)
for each symbol s_i ∈ O do
    for each terminal rule A → s_i in G do
        Calculate recognition probability P_rec(s_i)
        T[{s_i}].A = P(A → s_i) × P_rec(s_i)
    end
end

// Phase 2: Bottom-Up 2D Spatial Parsing (Non-terminals)
for subset size j = 2 to n do
    for every subset S ⊆ O where |S| = j do
        for every disjoint binary partition (S_1, S_2) of S do
            for each rule A → B C in G do
                for each spatial relation r ∈ R do
                    // Check if S_1 and S_2 are valid for B and C
                    if B ∈ T[S_1] and C ∈ T[S_2] then
                        // Calculate geometric penalty based on
                           bounding boxes
                        P_geo = EvaluateSpatialConstraint(S_1, S_2, r)
                        if P_geo > threshold then
                            prob = P(A → B C) × T[S_1].B × T[S_2].C × P_geo
                            // Keep the highest probability
                               derivation for this subset
                            if prob > T[S].A then
                                T[S].A = prob
                            end
                        end
                    end
                end
            end
        end
    end
end
```

The most notable feature of Algorithm 2 is the exponential explosion in search space, which is shown in Phase 2.

- **The Subset Loops (Lines 8-10):** Instead of checking linear lengths, Line 9 generates every possible combination of strokes. Line 10 then divides that subset into two disjoint groups (S_1 and S_2). For n symbols, the number of ways to partition subsets into two groups is 3^n. This is because here there are three possible scenarios. Either the a symbol falls into the the left hand side of the grammar rule so S_1, the right hand side of the grammar rule so S_2 or the symbol does not fall into any side of the grammar rule. This moves the time complexity from the polynomial $O(n^3)$ of 1D text to an exponential $O(3^n)$. [33]
- **The Spatial Relation Loop (Line 12):** For every valid grammar rule, the algorithm will test if the geometric relationship holds. It checks if S_2 is a Superscript of S_1, a Subscript, etc.
- **The Geometric Evaluator (Line 15):** This is the hard-coded constraint part. It calculates the bounding boxes of the strokes in S_1 and S_2 and returns a probability P_{geo}. If the symbols are aligned perfectly according to the rules of math provided, $P_{geo} \approx 1.0$. If they are misaligned, the probability drops exponentially, as a result pruning that branch of the parse tree.

Recent advancements in the HMER community point to switching to approaches that utilize soft constraints. These are approaches that make use of neural based concepts. The idea behind these are to assign probabilities to different interpretations of a handwritten equation based on the geometric rules and to maintain ambiguity of certain characters until the surrounding context can be used to decipher a specific interpretation [10].

1.2 Neural-Based Concepts

Neural based HMER's make use of modern neural based concepts in the machine learning field such as neural networks, transformers e.t.c. This section

will highlight the inspiration and intuition behind some of these neural based concepts.

1.2.1 An Overview of the Biological Neuron Structure and Function

Neural networks are heavily inspired by the human brain. The brain contains contains neurons and these are the basic working unit for it.

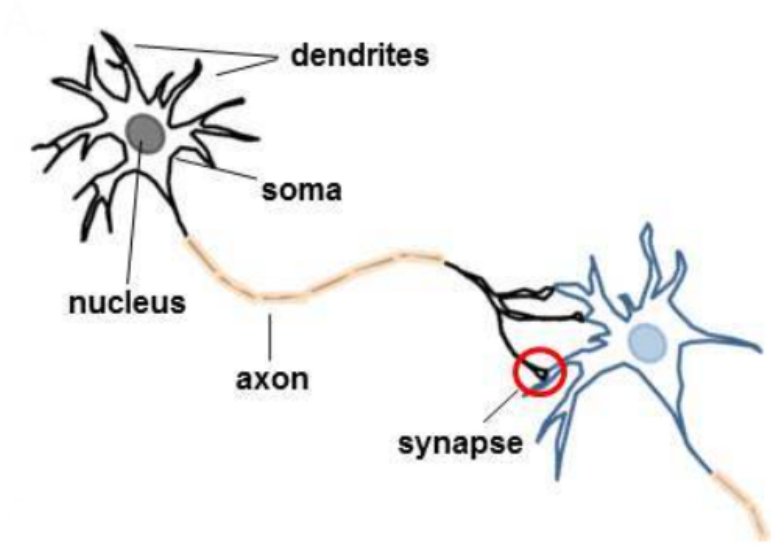

Figure 2: Structure of biological neuron [30]

To understand the principles used in neural networks, it will be beneficial to examine the four primary components of a neuron, namely, the dendrite, the soma, the axon and the soma.

- **Dendrites**: As shown in figure 2, these extend outward from the soma in a highly branched, tree like structure. They serve as a primary receptive field for the neuron by collecting incoming electro-chemical signals from other neighbouring neruons. The extent and branching of these dendrites, is what determines the amount of external stimuli a neuron can capture [36]
- **Soma**: This is the central processing unit of the neuron. It's main function is to aggregate excitatory and inhibitory electro-chemical signals,

which play a part in determining whether a neuron is fired or not respectively. This process is also followed by introducing a biological threshold. After the soma aggregates these signals, if the net aggregation or accumulation is greater than a specific membrane threshold , the neuron is triggered into an active state, more formally noted as the **firing of a neuron**. Conversely, if the net aggregation is not greater than this membrane thershold then the neuron remains at its resting potential, the baseline signal the neuron already had before anything happened to it, meaning that no signal is transmitted by the neuron. [36].

- **Axon**: If the signal aggregate manages to cross the membrane threshold, the neuron generates an all or nothing electrochemical impulse, known as an action potential. The axons main job is to take this signal and pass it down its elongated cable-like structure to act as the neurons output transmission line. The axon makes sure that this generated impulse travels far away from the soma and toward the next set of neurons [36].

- **Synapse**: This is where the transmission of the signal to the next neuron occurs. Shown by the red circle in figure 2, they are microscopic junctions located at the axon terminals which bridge the gap between the transmitting axon and the receiving dendrites of the next neuron[36].

It is important to note that the efficiency of signal transfer between neurons is not uniform. Synapses have a property known as synaptical plasticity which is the ability to dynamically change the strength of their connections. Depending on certain properties of signals, a synapse has the ability to either amplify or dampen future signals that follow the same path by modifying its connection strength to neighboring neurons.[36].

1.2.2 Artificial Neural Networks

Following the MP-neuron[6] was the **perceptron**. It extends the MP neuron by incorporating learnable weights, a bias and allowing for real valued inputs to be used.

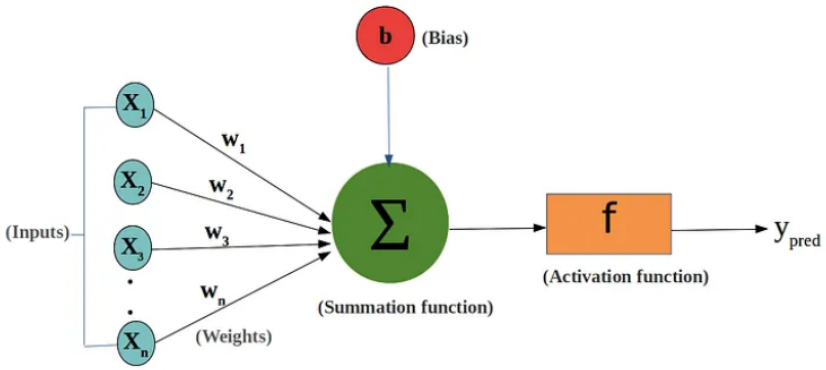

Figure 3: Perceptron Structure[13]

Given inputs $x_1, x_2, \ldots, x_n \in \mathbb{R}$, each input is associated with a weight $w_i \in \mathbb{R}$. The perceptron computes a weighted sum:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

where $b \in \mathbb{R}$ is the bias term. This bias term is linked to the threshold value from the MP-neuron in the sense that $b = -\theta$. The idea here is to make b learnable as well as all other weights.

The output is then determined using a step activation function:

$$y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

A key advantage of the perceptron is its ability to **learn** weights and biases from labelled data using the update rule:

$$w_i \leftarrow w_i + \eta(y_{\text{true}} - y_{\text{pred}})x_i$$
$$b \leftarrow b + \eta(y_{true} - y_{pred})$$

where η is the learning rate[27]. The idea is to continuously apply these update rules , while there is still a misclassified point.

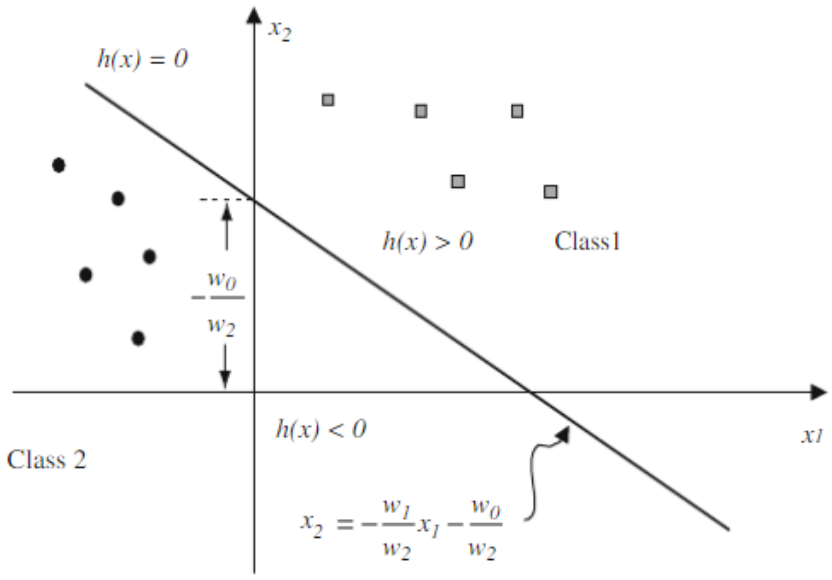

Figure 4: Perceptron decision boundary example in 1d[9]

Figure 4 shows an example of applying the update rules to a dataset where $w_0 = b$. It also shows an alternate form of writing the decision boundary in 1d. Visually speaking, because we have now allowed for learning and updating of weights and the bias, the perceptron has the ability to rotate the boundary , which stems from the $-\frac{w_1}{w_2}$ part as it can be viewed as modifying the gradient of the line, and also shift the decision boundary up or down which implicitly moves the boundary left or right, which stems from the $-\frac{w_0}{w_2}$ part as it can be viewed as modifying the y-intercept.

Similar to the MP-neuron, the perceptron does not work well with linearly inseparable problems. Overcoming this limitation required the development of the Multilayer Perceptron (MLP), a fully connected feedforward architecture that introduces intermediate processing stages[24].

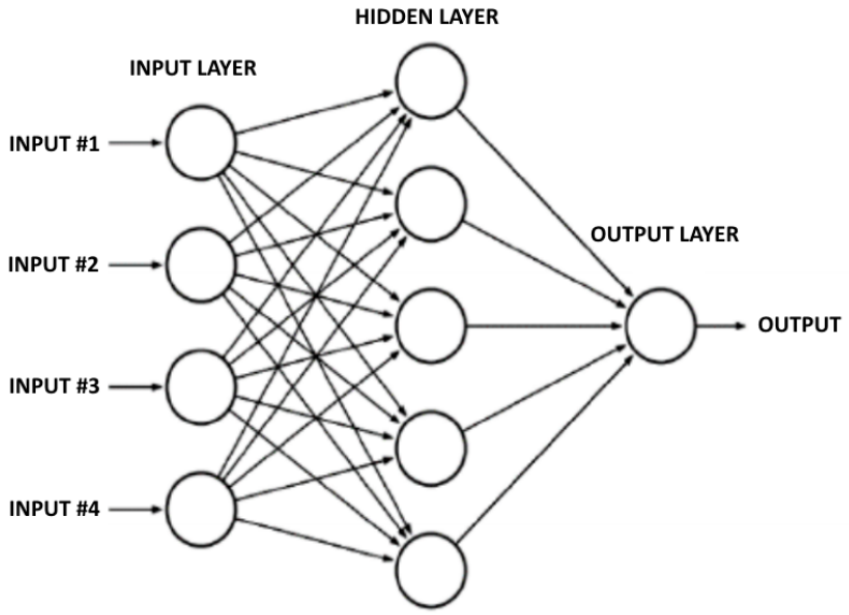

Figure 5: Multi layer Perceptron Structure Example[11]

A multi layer perceptron consists of three different types of layers, namely, an input layer that receives the initial data vector, one or more hidden layers for intermediate calculations and an output layer that produces the final prediction. The idea is to stack perceptrons over each other per layer, and connect them to neighboring perceptrons i.e perceptrons in subsequent layers.

Mathematically, let $\mathbf{a}^{(l-1)}$ represent the vector of activation outputs from the previous layer (where $\mathbf{a}^{(0)}$ is the initial input vector $\mathbf{x}$). The pre-activation vector, $\mathbf{z}^{(l)}$, for the current layer l is calculated using the weight matrix $\mathbf{W}^{(l)}$ and the bias vector $\mathbf{b}^{(l)}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

This implementation works better with linearly inseparable problems. [28].

1.2.2.1 Activation Functions

An activation function is a mathematical function attached to each neuron, and decides if a neuron should be fired or not. It can also help normalise the

output of each neuron. This idea can be more generally viewed as

$$y = f(z)$$

where

- z is the pre-activation value
- f is the activation function itself
- y is the activation value

Sigmoid Activation Function:
The sigmoid activation function is a non-linear activation function denoted as

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function is especially strong in binary classification tasks where we need to see if points in a dataset belong to a specific class.

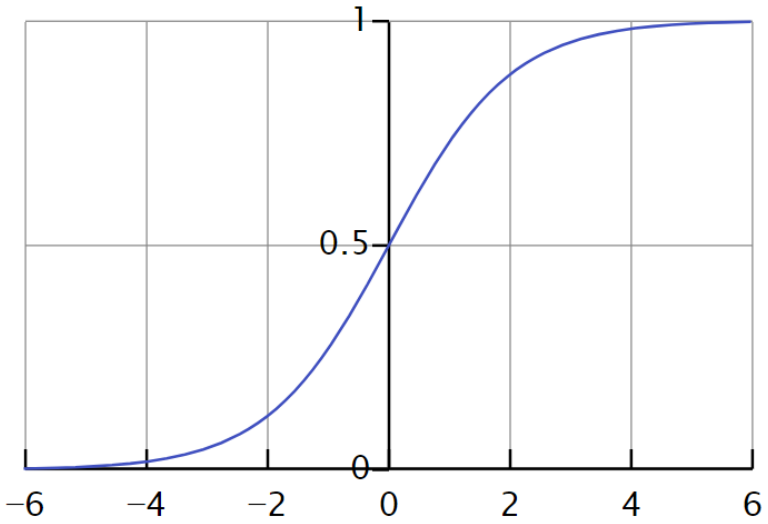

Figure 6: Sigmoid function graph

If we tried to use a linear activation function for classification tasks it could work, however when training the neural network, it would be beneficial to measure the deviation of the predicted value from the true label the point

should have. Since linear activation functions are unbounded, the measurement of the deviation from the true value is also unbounded. The sigmoid function makes use of a concept called confidence. The graph shown in figure 6 shows the sigmoid function. All of it's outputs are squashed between 0 and 1 , as a result if predicted values are above 0.5 for example , we can interpret that as the model saying it is more confident that a point belongs to one class than the other. This is very useful as binary labels in practice are 0 and 1. However, one major disadvantage of the sigmoid function is that it makes the neural network suffer from the vanishing gradient problem[15]. When using gradient descent to train the neural network through backpropagation it involves multiplying multiple gradients of activation functions together. The maximum possible gradient the sigmoid function can have is 0.25 hence as a result, in very deep neural networks containing these , as we multiply multiple 0. values together, the gradient tends to 0.

Tanh Activation Function:
The tanh activation function is defined as

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

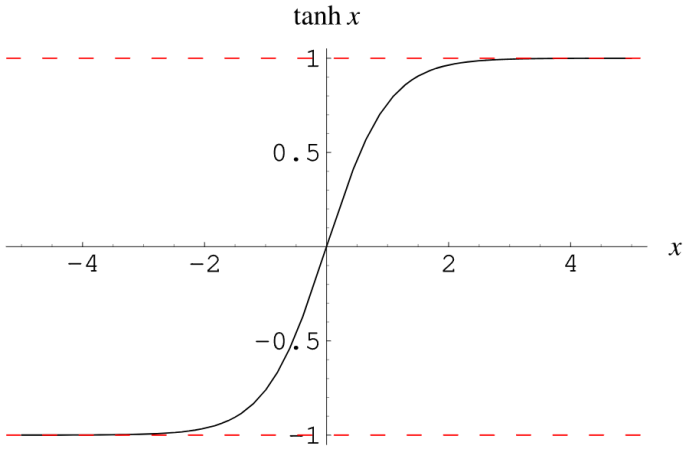

Figure 7: Tanh Function

It's graph is shown in figure 7. It is a transformed version of the sigmoid function. They both have the same benefits but there is one important thing to note about this function. In terms of the vanishing gradient problem, the tanh

function is actually better than the sigmoid function and this is due to the fact that the maximum gradient this function can yield is 1. However it still suffers from the vanishing gradient problem in deep-neural networks.

ReLU Activation Function:
The ReLU function is is also another non-linear activation function. It is defined to be,

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

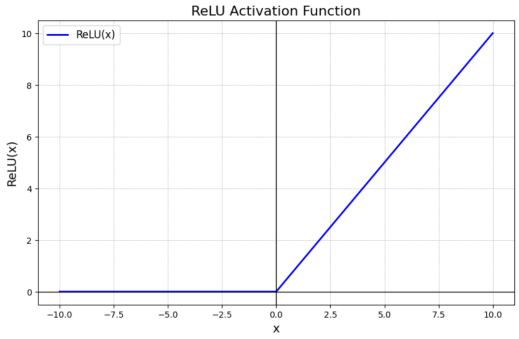

Figure 8: ReLU Function

It's graph is shown in figure 8. Notice that for any positive input its gradient is 1 and 0 otherwise. This implies that the ReLU activation function does not suffer from the vanishing gradient problem. However, neural networks made out of ReLU activation functions , tend to suffer from the dying ReLU problem[22]. This is when the ReLU neuron either refuses to learn or learn slowly. In a case where currently, the models has updated its weights to be extremely small numbers and the bias to be highly negative i.e, $z = \mathbf{w}_{small}^T \mathbf{x} + b_{high^-}$.Now say it gets some input. No matter what , that input gets crushed down by the small weights and because bias is extremely negative, z gets dominated by it. So now z will always be highly negative, no matter the input is. We know this is what gets fed into the ReLU activation function itself and so when it is time to compute the gradient based on this, it will always be 0. This means that , even after more iterations, the neuron this is tied to will never get updated because the gradient will be 0 and thus the gradient descent update during backpropagation will always get the gradient a gradient of 0 meaning weights and biases don't get updated.

Softmax Activation Function:
The softmax activation function is a bit different from traditional activation functions talked about before. Unlike the activation functions mentioned before, which take a single value, the softmax function takes as input a vector z of N real values, and normalizes it into a probability distribution consisting of N probabilities proportional to the exponents of the input values. Mathematically, it is defined as

$$y_i = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

Softmax is often used for multi-class classification tasks. Softmax is mainly beneficial during the training of a neural network. It serves as a way to bring the predicted distribution closer to the one-hot true distribution in practice

1.2.2.2 Loss Functions

A loss function is used to determine the deviation of a models predicted values from the true values. These are especially useful when training models as it enables the model to correct it's weights and biases to better **minimize** the error of the loss function it uses. There are many loss functions but we will take a look at two specific loss functions, relevant in the context of HMER for this project, namely Cross-Entropy (Log loss) and Mean Squared Error (MSE)

Cross-Entropy:
Cross Entropy loss (for a binary classification task) is defined as

$$L = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

where

- y is the actual class
- $\hat{y}$ is the predicted output between 0 and 1

Cross Entropy can actually be extended to an N class classification task. Let $y = y_1$ and $1 - y = y_2$. We notice that the binary cross entropy loss formula becomes

$$L = -[y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2]$$

with

- $y_1 + y_2 = 1$
- $\hat{y}_1 + \hat{y}_2 = 1$

Notice that we effectively multiply the actual value by the log of the probability for each class. This means that we can extend this idea to multiple classes

$$\implies L = -\sum_{i=1}^{N} y_i \log \hat{y}_i$$

with with,

- $\sum_{i=1}^{N} y_i = 1$
- $\sum_{i=1}^{N} \hat{y}_i = 1$

This is what will be used to determine how far off the HMER is from the correct mathematical symbol when it is being trained.

Mean Squared Error:
The Mean squared Error loss function is defined as

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y)^2$$

This is a relatively simple function as it just averages the squared difference between actual and predicted values. It's strength lies in its ability to penalise large errors more heavily than smaller ones. Additionally, this will be used to determine how far off the model is from the true count of symbols when we introduce CAN to the basline model.

1.2.2.3 Gradient Descent and Backpropagation

Gradient descent is an iterative algorithm used, most commonly, in the context of minimizing loss functions. To understand why gradient descent has the ability to do this we first start by examining the formula $\nabla f_{x_0} \cdot \mathbf{u}$ which is

used to find the slope of a function in any dimension in the direction $\mathbf{u}$, where $\mathbf{u}$ is a unit vector and ∇f_{x_0} is the slope of the function at a specific point. This can also be expressed as $\nabla f_{x_0} \cdot \mathbf{u} = |\nabla f_{x_0}||\mathbf{u}| \cos\theta = |\nabla f_{x_0}| \cos\theta$. We observe that this formula is maximized i.e we get the highest slope when $\theta = 0^\circ$. This also means that the angle between the direction vector and gradient vector are the same meaning we move in the direction of the gradient vector, ultimately meaning that the gradient vector yields the direction of steepest ascent from any point. This leads to the gradient descent update rule,

$$x_{t+1} \leftarrow x_t - \alpha \nabla f_{x_t}$$

which goes in the opposite direction to get the steepest descent path from a specific point on a loss function.

Backpropagation leverages this idea of gradient descent to minimize loss functions. Since loss functions depend on $z = w \cdot x + b$ we can define the backpropagation update rule for the weights and biases of a neural network as

$$w_{t+1} \leftarrow w_t - \alpha \nabla L_{w_t}$$
$$b_{t+1} \leftarrow b_t - \alpha \nabla b_{w_t}$$

1.2.2.4 Convolutional Neural Networks(CNN)

When it comes to image processing, artificial neural networks alone lead to some problems. The traditional way of processing images via neural networks involves stacking all pixel values together for computations. This can be very problematic, especially for relatively huge images. CNN's are used to mitigate this. Pooling is one of the techniques used, which downsample the dimensions of the feature maps.

In traditional CNNs, local spatial patterns are learned when sliding learnable filters through the input image. In a feed-forward CNN, the output $\mathbf{z}^{(l)}$ of the l^{th} layer is computed by applying a non-linear composite function $H^{(l)}(\cdot)$ (Batch Normalization, Rectified Linear Unit and Convolution) only to the output of the previous layer: $\mathbf{z}^{(l)} = H^{(l)}(\mathbf{z}^{(l-1)})$. While stacking these layers will allow the network to learn more complex hierarchies of features, very deep CNNs are affected by the vanishing gradient problem and the loss of spatial

resolution during multiple pooling operations.

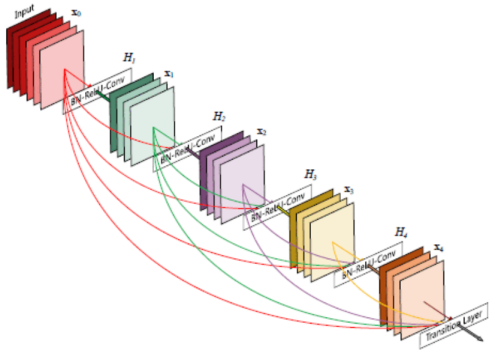

Figure 9: DenseNet Connection[18]

DenseNet is a type of CNN which fundamentally alters the connectivity pattern of a traditional CNN. Instead of only drawing inputs from the immediate previous layer, each layer in a DenseNet receives the feature maps from all preceding layers within a dense block. Mathematically, if we denote the concatenation of feature maps produced by layers 0 to $l-1$ as $[x_0, x_1, \ldots, x_{l-1}]$, the l^{th} layer in a DenseNet computes its output as:

$$x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$$

It is also important to note that between each layer , batch normalization is done. This is a way to fix the mean and variance for each layers output. This has shown to increase the convergence rate of models when being trained. As a result of this, training deeper models becomes significantly more stable and ultimately alleviates this vanishing gradient problem[18][19].

1.2.2.5 Recurrent Neural Networks

When dealing with sequential data, standard artificial neural networks and CNNs have many limitations. They are based on the idea that all inputs and outputs are independent and normally have fixed-size input vectors. However, for language generation, the sequential order of elements are important. Recurrent neural networks (RNNs) are used to overcome this. RNNs offer a hidden state that acts as internal memory and allow the past to be kept

alive in sequential time steps.

In a traditional RNN, the temporal patterns are learned by applying a transformation over the sequence. At a given time step t, the network computes the current hidden state h_t by applying a nonlinear activation function $\sigma(\cdot)$ (typically tanh) to a linear combination of the current input x_t and the previous hidden state h_{t-1}:

$$h_t = \sigma(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$$

. However, during Backpropagation Through Time (BPTT), the gradients are repeatedly multiplied by the recurrent weight matrix W_{hh}. It has been proven that if the largest eigenvalue of this matrix is less than 1 then the gradient decays exponentially, leading to the vanishing gradient problem. Conversely, if the eigenvalue of this matrix is greater than 1 then the gradients grow unboundedly leading to the exploding gradient problem. [2] This shows that for longer sequences , RNN's will tend to fail.

The Gated Recurrent Unit (GRU) is a type of RNN that alters the connectivity to preserve and capture longer sequences better. Instead of a simple single-layer transformation, GRU uses gating mechanisms, including update gate z_t and reset gate r_t, to control the information flow. The last hidden state h_t is computed as a linear interpolation between the previous state h_{t-1} and the new candidate state $\tilde{h}_t$.:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Because this update rule relies on addition and multiplication rather than only multiplication, this alleviates the vanishing gradient problem[3]

2 HMER and Math Engine Implementation

2.1 MathWriting Dataset and Pre-processing

2.1.1 MathWriting Dataset

The MathWriting Dataset was chosen specifically for this project because of the robust and huge number of samples in it . The data split between test, training and validation is also already done in the dataset.

In addition there is a synthetic subdataset in it. Human contributors were not just limited in number but also by the screen sizes of their phones or tablets,in which they wrote equations on. This restricted the length of the equations they could write down and also the amount of data that could be collected. The synthetic subdataset fixes this. It is a dataset made by artificially stitching together different symbols written by different humans. This allowed for more data as a whole and even longer equations to be created.

```
<ink xmlns="http://www.w3.org/2003/InkML">
  <annotation type="label">\phi^{*}(T^{*}N)\rightarrow T^{*}M</annotation>
  <annotation type="splitTagOriginal">train</annotation>
  <annotation type="inkCreationMethod">human</annotation>
  <annotation type="sampleId">00002504391b73b5</annotation>
  <annotation type="normalizedLabel">\phi^{*}(T^{*}N)\rightarrow T^{*}M</annotation>
  <traceFormat>
    <channel name="X" type="decimal" />
    <channel name="Y" type="decimal" />
    <channel name="T" type="decimal" units="ms" />
  </traceFormat>
  <trace id="0">113.94 82.76 0.0,109.94 85.26 23.0,105.45 88.76 35.0,101.63 92.57 44.
  <trace id="1">135.93 67.78 631.0,133.93 73.27 675.0,132.64 76.50 682.0,128.93 86.76
  <trace id="2">217.89 38.80 1166.0,213.39 43.30 1211.0,208.39 48.29 1222.0,202.89 53
  <trace id="3">183.90 53.79 1421.0,186.91 53.80 1475.0,189.90 59.28 1490.0,194.40 65
  <trace id="4">164.91 67.78 1713.0,169.03 68.05 1751.0,173.91 67.78 1758.0,179.88 65
  <trace id="5">235.88 96.75 2749.0,241.87 97.75 2794.0,245.78 97.75 2803.0,253.87 97
  <trace id="6">265.36 101.74 7582.0,265.86 105.74 7626.0,265.36 109.74 7638.0,265.36
  <trace id="7">353.32 70.77 8280.0,351.32 74.77 8313.0,348.32 78.27 8325.0,345.32 82
  <trace id="8">322.83 82.76 8535.0,327.33 81.76 8592.0,331.33 82.76 8604.0,334.47 84
  <trace id="9">327.33 91.25 8920.0,332.74 91.18 8975.0,338.82 90.75 8989.0,344.82 91
  <trace id="10">381.30 179.17 9851.0,383.80 173.18 9920.0,385.19 168.82 9941.0,385.8
  <trace id="11">267.36 57.78 11156.0,263.56 59.58 11200.0,260.36 63.28 11212.0,256.3
  <trace id="12">457.76 55.79 11959.0,465.76 61.78 11992.0,468.76 70.77 12004.0,470.6
  <trace id="13">508.74 123.72 12821.0,512.73 123.22 12877.0,516.47 123.22 12906.0,51
  <trace id="14">669.15 91.25 13974.0,672.34 91.12 14010.0,677.15 90.75 14018.0,684.7
  <trace id="15">700.64 89.76 14229.0,701.63 93.75 14274.0,702.12 100.08 14286.0,702.
  <trace id="16">777.59 58.78 14800.0,775.60 62.28 14833.0,772.10 70.27 14845.0,768.3
  <trace id="17">750.61 64.78 15057.0,754.11 67.28 15090.0,757.11 70.77 15101.0,760.1
  <trace id="18">743.11 81.76 15394.0,748.14 81.76 15424.0,756.11 81.26 15439.0,764.1
  <trace id="19">787.09 167.18 16279.0,791.02 167.33 16320.0,795.59 163.19 16347.0,79
</ink>
```

Figure 10: MathWriting Dataset Sample

Typically each sample in the dataset follows a fixed format.Figure 10 shows an example of a sample of this. Each item encased in " <> " is formally referred to as a tag. The tags that were necessary in the scope of the project are,

- **Label Tag** - Shown on line 2 in figure 10, this is the LaTeX format of the sample we are considering. So in figure 10 the LaTeX format is $b_{n+1} = \max(a_n, b_n + 1)$
- **NormalizedLabel Tag** - Shown on line 7 in figure 10, this is the underlying **ground truth** of the sample considered. This is different from the label tag. As there are many different ways to write a mathematical expression in LaTeX format that will render to images that are equivalent in handwritten form, MathWriting applied a series of transformations to eliminate some of these variations while retaining the same meaning. This greatly improves the performance of models and made their

evaluation for handwritten equation recognition more precise [14].

- **TraceFormat tag** - Shown from lines 8 to 12 in figure 10, this highlights the structure of each recorded stroke. The idea is to record sequences of touch points (coordinates) obtained from a touch screen or digital pen in this format[14]. The "X" , defines the x-coordinate and the "Y", defines the y-coordinate. The "T" in ms, defines the time at which the touch point was recorded. The "T", however, was not necessary as it is more useful for online or real-time handwritten equation recognition systems. This project mainly focuses on static images that do not change overtime. So overall each touch point follows the format of (x, y, t)
- **Trace tag** - An example of one is shown on line 13 in figure 10. A trace tag highlights all recorded touch points in a single stroke of the pen. It contains multiple touch points as shown which are delimited by commas. Each point follows the structure of (x, y, t) as said before.

2.1.2 Pre-processing

2.1.2.1 Converting dataset to images

Algorithm 3: Main Pipeline: Preprocess InkML Dataset

Input: *root*: Path to raw InkML data directory, *out*: Path to save processed images and ground truth

Output: Saved PNG images and a CSV file

$data \leftarrow \emptyset$ `// Initialize empty list for metadata mapping`

$paths \leftarrow$ Sorted list of all $*.inkml$ files in *root*

for ***each*** $p \in paths$ **do**

  Attempt to extract and rasterize data:

  $strokes, label \leftarrow \text{ReadInkML}(p)$

  if *extraction is successful* **then**

    $img \leftarrow \text{RasterizeStrokes}(strokes)$

    Save *img* as PNG to *out*

    $data \leftarrow data \cup \{(p.\text{name}, label)\}$

  end

end

Save *data* to CSV file in *out*

Algorithm 4: Helper function to Parse InkML File

Input: *path*: Path to a single InkML file

Output: *strokes*: List of arrays containing (x, y) coordinates, *label*: Ground truth string

$root_{xml} \leftarrow$ Parse XML tree from *path*
traces $\leftarrow$ Find all $\langle trace \rangle$ nodes in $root_{xml}$
strokes $\leftarrow \emptyset$
for ***each*** *trace* $\in$ *traces* **do**
    points $\leftarrow \emptyset$
    coords $\leftarrow$ Extract and split text inside *trace*
    for ***each*** (x, y) *pair in coords* **do**
        points $\leftarrow$ *points* $\cup \{(x, y)\}$
    end
    if *points* $\neq \emptyset$ **then**
        strokes $\leftarrow$ *strokes* $\cup \{points\}$
    end
end
ann $\leftarrow$ Find normalized label or truth annotation node
label $\leftarrow$ Extract text from *ann*
return *strokes*, *label*

Algorithm 5: Helper function to rasterize Strokes to Image

Input: *strokes*: List of coordinate arrays, $stroke\_width = 2$, $margin = 2$, $target\_h = 128$, $max\_w = 768$

Output: *img*: Rasterized image matrix

if *strokes is empty* **then**
  return White image of size $target\_h \times target\_h$
end
$all\_points \leftarrow$ Concatenate all *strokes*
$min_x, min_y \leftarrow$ Minimum coordinates across all_points
$max_x, max_y \leftarrow$ Maximum coordinates across all_points
$w \leftarrow (max_x - min_x) + 2 \times margin$
$h \leftarrow (max_y - min_y) + 2 \times margin$
$img \leftarrow$ White matrix of size $w \times h$
$offset \leftarrow (min_x - margin, min_y - margin)$
for *each* $s \in strokes$ **do**
  $s_{shifted} \leftarrow s - offset$
  Draw anti-aliased lines for $s_{shifted}$ on img
end
$scale \leftarrow target\_h / h$
$new\_w \leftarrow w \times scale$
$img \leftarrow$ Resize img to $(new\_w, target\_h)$
if $new\_w < max\_w$ **then**
  Pad img on the right with white pixels to reach max_w
end
else
  $img \leftarrow$ Downscale img width to max_w
end
return img

Algorithms 3 to 5 show all of the work done in converting the mathwriting dataset into images. The main idea is to connect all given touch points per trace tag. However before this step, it will be beneficial to find a way to access each inkml sample and hence each tag programmatically. Algorithm 4 shows this. It takes an argument, namely "path", this is the path to a single inkml sample. In this file, it finds all trace tags and extracts each individual touch point per trace tag. The results are stored in a list called strokes. So strokes, is

a list of lists, where each innate list, is the series of touch points for each trace. As said before, the time aspect of the tags are dropped. The final part of this algorithm simply finds the normalized label for the sample considered which is always in the annotation tag, refer to figure 10.

The next step is to pass the strokes for the sample to rasterize strokes function shown in algorithm 5. Due to the horizontal nature of long mathematical equations, the idea is to draw the equations out on a rectangular strip. The "target_h" and "max_w" properties show this. MathWriting stated that as the samples were taken on different devices, the absolute coordinates in the trace tags will vary in terms of maximum and minimum values[14], hence the first step is to get the pair of coordinates with lowest and highest values to gauge the maximum needed height and width to fully display the equation. Relative to the margin, the points are then drawn on the img variable which is a matrix of ones (which represents fully white) by placing all points on it then connecting them. In order to meet the 128×768 constraint, it was necessary to compute a scale, which gives us how much we need to scale the current image to meet the targeted dimensions while still preserving aspect ratio. After , this resizing the image based on scale was carried out. The new width computed could still be either above or below the maximum width so based on whichever case the image falls in, it is either padded with extra white background or downscaled as a whole.

The last step brings both functions spoken about before together. This is shown in figure algorithm 3. It first goes into the root path and finds all inkml files. A for loop is then used to loop through these files while rasterizing each one and storing each samples respective ground truths in a csv file called data.

2.1.2.2 Building a vocabulary and custom dataset

In addition to the above pre-processing steps and since we the final model is to predict latex symbols a vocabulary is necessary. HMER is inherently a natural language processing (NLP) problem and NLP problems always need these. A vocabulary is formally known as a set of unique or allowed words or symbols that a model is allowed to output. In this case , the vocabulary will consist of LaTeX symbols.

In order to create a vocabulary the most logical step is to find a way to extract individual LaTeX symbols from the MathWriting dataset. To do this, the data CSV file which mapped images to ground truth was used alongside a tokenizer for symbol extraction. The idea was to loop through the training, synthetic and symbol dataset (as these were used for training) and use this tokenizer to tokenize each mathematical ground truth sequence. The tokenizer used the regex "findall" function with the pattern r"\\[a-zA-Z]+|c\\[^a-zA-Z]|." to extract unique symbols from each sequence.

- The first part is responsible for catching tokens like \frac, \phi, \in e.t.c
- The second part is responsible for catching tokens like \#, _
- The third part is responsible for catching tokens like a, 2, (, e.t.c. It is important to note that this can also catch whitespaces, so an extra step for stripping the returned list of tokens of whitespaces is done.

Following this it is necessary to add 4 special tokens namely "<SOS>" depicting start of sequence, "<EOS>" depicting end of sequence , "<PAD>" and "<UNK>" depicting unknown, to the vocabulary. The <EOS> token enables the model to define a probability distribution over sequences of variable lengths. The <SOS> token serves as the initial trigger state for the decoder itself.There are lot's of niche symbols in mathematics and so to handle out of vocabulary tokens, the <UNK> token is necessary[31][20][16]. All tokens are stored in a python dictionary for use.

Lastly for this section a custom MathDataset class was created which also inherited pythons dataset class. There were two main reasons for this. The first reason is because WAP cannot read images or strings directly. They require tensors. So the job of the dataset is to get the file paths, convert the images into tensors, and map the string characters to their corresponding integer vocabulary IDs. The second reason is because the MathWriting dataset is huge. If we attempt to load all of these images into the computer at once. The system could crash. Since this custom dataset also inherits pythons dataset class, we can make use of lazy loading to only load in images by batches during training.

2.2 Watch, Attend Parse (WAP)

One of the newer approaches to hand written mathematical equation recognition frameworks the Watch Attend Parse (WAP) model.This approach will be the baseline implementation for this project and further improvements on the resulting architecture will be made.

The WAP is an attention based encoder-decoder model. Encoder-decoder models are neural networks , used for processing sequential data as well as generating them[26]. The WAP model is made up of 2 parts, namely the "Watcher" and the "Parser".

- **The Watcher** - The watcher is a convolutional neural network (CNN) encoder that maps ME images to high-level features. For the project itself, a deep FCN architecture will be adopted as the encoder here to deal with the much larger variation in the size of the math symbols in handwritten expressions than in the machine-printed ones with much smaller model footprint. It is also observed from experimental results that the deep FCN architecture performs much better for HMER task than any other types of CNN's. [12]
- **The Decoder (Parser)** - The parser is a recurrent neural network (RNN) decoder that converts high-level features received from the watcher into output sequences, word by word. The "attention" part of the model is implicitly in the parser. Basic attention mechanisms maintain a 2d feature map for the HME image that is to be recognised. Attention mechanisms like this made it such that the HME images no longer have to be squashed down, as earlier models did. However, basic attention models still have a major flaw when it comes to HMER which is a lack of coverage . Essentially, this lack of coverage is referring to the attention mechanism's failure to track which parts of the image have already been processed and as a result, because some parts look more relevant, they get processed multiple times or the mechanism skips over certain parts as a result because it doesn't realize some parts have

not been processed yet. A coverage vector was used to attack this problem. This is taken as an extra input when the model looks at the next symbol. If the values in it are high when it looks at the next symbol then the model effectively realises that it has viewed that pixel already.[12]

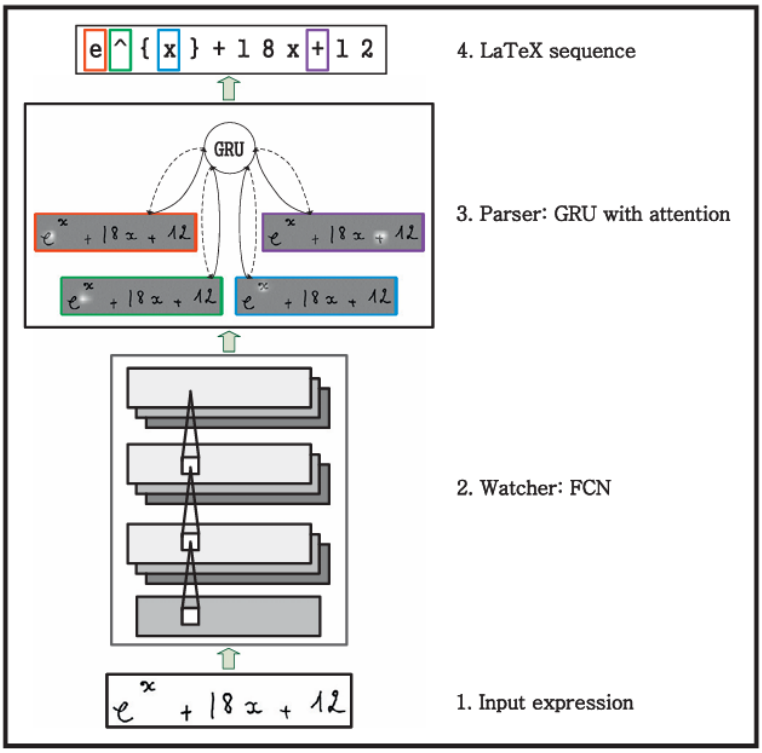

Figure 11: WAP neural network structure

This Watcher and Parser are trained together. As implied by figure 11, the parser has the ability to give feedback to the watcher to extract better features during backpropagation because of it's direct connection to it, while the watcher helps the parser understand where to look next during forward propagation.

2.2.1 The Watcher FCN

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

Figure 12: DenseNet steps[18]

The FCN architecture used for the watcher the denseNet. Specifically the denseNet-121 as it has the cheapest computational costs while still boasting very good performance. To skip having to train the denseNet again to recognize images the default weights of the DenseNet-121 architecture were used. Figure 12 details the steps that an image will go through in a denseNet. The DenseNet-121 column shows exactly what will happen to the 128×768 image as it passes through this network. It is important to note that handwritten equations preprocessed were all grayscale images, however denseNet expects RGB images. To account for this, grayscale images were converted into RGB images. All this meant is that we would have 3 identical channels for the image which are fed into the watcher. Figure 12 shows the respective pooling and convolution steps used at each stage as the image passes through the denseNet. For context, after the image passes through the dense block (4) it has dimensions [1024, 4,22], where 1024 represents the number of channels of the image. What all this as a whole represents for the HMER is a grid of the original input image. Each location within this grid, maps back to a contiguous overlapping region in the original image, known as a receptive field. Each contiguous location is described by a vector of length 1024. So we are no longer looking at raw pixel intensities alone, rather, we are now looking at a vector that quantifies the prescence of features such as maths symbols and the spatial relationship between these symbols (like subscripts or fractions) within a specific area of the image.

Before moving onto the parser, it is important to note that the RNN in the parser cannot read the 3D output we have. So instead we flatten these dimensions by defining:

$$\mathbf{A} = \{\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_L\}$$

where

- $\mathbf{a}_i \in \mathbb{R}^D$ is the 1024-dimensional vector representing a receptive field.
- $L = H \times W$, which in our case is 4×22 meaning we have 88 distinct receptive fields.

2.2.2 The Parser

2.2.2.1 Attention Mechanism

Classic attention machanisms aim to calculate an alignment score $e_{t,i}$ for every receptive field as:

$$e_{t,i} = \mathbf{v}^T \tanh(\mathbf{W}\mathbf{s}_{t-1} + \mathbf{U}\mathbf{a}_i)$$

where,

- $\mathbf{v}, \mathbf{W}$ and $\mathbf{U}$ are learnable weights.
- $\mathbf{s_{t-1}}$ is the parsers hidden state or current memory. This is used for LaTeX grammatical correctness in this case. For example if the list of sequences that it has predicted ended with { it will know to look for a }

Intuitively speaking, $\mathbf{W}\mathbf{s}_{t-1}$ allows the model to know what it needs next (grammatically) while $\mathbf{U}\mathbf{a}_i$ summarizes what structural shapes are inside the receptive field. This means that adding both of these allow the model to see how well "alligned" what it needs next is with what is currently available in it's receptive field. If both values are close then the $\mathbf{v}$ vector is used to try to match with the values in the addition. For example if values are highly negative in there (reflecting that both $\mathbf{U}\mathbf{a}_i$ and $\mathbf{W}\mathbf{s}_{t-1}$ were both higly negative) , it will also have highly negative values to balance it out to make it positive to reflect

high alignment. We then turn these raw scores into probabilities, by applying the softmax function. This gives us the attention weights $\alpha_{t,i}$:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{L} \exp(e_{t,j})}$$

As said before, however, there is a necessity for the coverage vector to prevent the model from over-parsing. The idea is to multiply each 1024-dimensional vector by its attention weight and sum them all together to create a single, focused Context Vector ($\mathbf{c}_t$) for a specific time step:

$$\mathbf{c}_t = \sum_{i=1}^{L} \alpha_{t,i} \mathbf{a}_i$$

This enables the model to know which areas it has spent the most time parsing during a specific time step. As a result we can redefine the attention mechanism now as:

$$e_{t,i} = \mathbf{v}^T \tanh(\mathbf{W}\mathbf{s}_{t-1} + \mathbf{U}\mathbf{a}_i + \mathbf{W}_c c_{t,i})$$

and update the coverage vector on the fly using attention weights at each time step.

Even with this there is a slight problem. As established before and even stated by Glorot et al [15], in this case, the tanh activation function can lead to the vanishing gradient problem. So instead it was decided to adopt Glorot et al's optimization for this by swapping tanh out with the ReLU function.

$$\implies e_{t,i} = \mathbf{v}^T \text{ReLU}(\mathbf{W}\mathbf{s}_{t-1} + \mathbf{U}\mathbf{a}_i + \mathbf{W}_c c_{t,i})$$

2.2.2.2 The Decoder

The decoder is a gated recurrent unit and this is used to produce the actual LaTeX token. At each time step t, the input to this is made up of the concatenation between the context vector and previously decoded token. This input token is initially represented as its location in the vocabulary, but because numbers like this cannot be directly passed into the gru (because these values could imply false mathematical relationships), this location ID is instead

passed into a learnable embedding layer. This layer acts as a lookup table that maps the ID to a higher dimensional space that represents the structure of the it's semantic and syntactic properties. Additionally, to avoid overfitting to specific tokens, a dropout layer is applied to the embedded vector produced[29]. The last input to the GRU is the previous hidden state. Using all this together, the output from here follows a linear transformation into the vocabulary space to get the z values for each possible token by creating a full connection between the GRU layer and the output layer.

2.2.3 Training the WAP model

Algorithm 6: HMER Single Batch Training Step

Input: Images $\mathbf{I}$, Target Sequence $\mathbf{Y}$ of length T, Encoder E, Decoder D, Optimizer, Criterion $\mathcal{L}$, Teacher Forcing Ratio ρ

$optimizer$.zero_grad()
$Loss \leftarrow 0$
$valid\_steps \leftarrow 0$
$\mathbf{A} \leftarrow E(\mathbf{I})$ `// Extract feature maps`
$\mathbf{s}_0 \leftarrow \mathbf{0}$ `// Initialize decoder hidden state`
$\mathbf{c}_0 \leftarrow \mathbf{0}$ `// Initialize coverage vector`

for $t = 1 \dots T-1$ **do**
  $r \sim U(0,1)$ `// Sample random probability`
  if $t = 1$ ***or*** $r < \rho$ **then**
    $\mathbf{y}_{t-1} \leftarrow \mathbf{Y}_{t-1}$ `// Teacher Forcing, so use ground truth`
  else
    $\mathbf{y}_{t-1} \leftarrow \arg\max(\hat{\mathbf{y}}_{t-1})$ `// Use own previous prediction`
  end
  $\hat{\mathbf{y}}_t, \mathbf{s}_t, \alpha_t, \mathbf{c}_t \leftarrow D(\mathbf{y}_{t-1}, \mathbf{s}_{t-1}, \mathbf{A}, \mathbf{c}_{t-1})$
  target_token $\leftarrow \mathbf{Y}_t$
  if *target_token* $\neq$ *PAD_IDX* **then**
    $step\_loss \leftarrow \mathcal{L}(\hat{\mathbf{y}}_t, \text{target\_token})$
    $Loss \leftarrow Loss + step\_loss$
    $valid\_steps \leftarrow valid\_steps + 1$
  end
end

$\overline{Loss} \leftarrow Loss / valid\_steps$ `// Normalize by valid tokens`
$\overline{Loss}$.backward() `// Backpropagation Through Time`
ClipGradients$(E, D, \text{max_norm} = 5.0)$ `// Prevent gradient explosion`
$optimizer$.step()
return $\overline{Loss}$

Algorithm 7: HMER Model Training Loop

Input: Custom MathDataset Loader $\mathcal{D}$ loading a batch of images $\mathbf{I}$ and ground truths $\mathbf{Y}$, Encoder E, Decoder D, Total Epochs N

Initialize E weights with defaults

Initialize D weights randomly

Initialize Optimizer Adam with learning rates η_E , η_D

for $epoch = 1 \dots N$ **do**

  E.train()

  D.train()

  $EpochLoss \leftarrow 0$

  $\rho \leftarrow \max(0.2, 1.0 - (epoch \times 0.1))$ `// Decay of Teacher Forcing`

  for ***each*** *batch* $(\mathbf{I}, \mathbf{Y}) \in \mathcal{D}$ **do**

    $batch\_loss \leftarrow \text{TrainStep}(\mathbf{I}, \mathbf{Y}, E, D, Optimizer, \mathcal{L}, \rho)$

    $EpochLoss \leftarrow EpochLoss + batch\_loss$

  end

  $\overline{EpochLoss} \leftarrow EpochLoss/|\mathcal{D}|$

  SaveCheckpoint$(epoch, E, D, Optimizer, \overline{EpochLoss})$

end

For a generic NLP problem the loss formula used , uses, cross-entropy[31]:

$$L = -\sum_{t=1}^{T} \log P(y_t \mid \mathbf{y}_{<t})$$

where

- y_t denotes the correct token
- $\mathbf{y}_{<t}$ denotes the set of previous tokens before the current time step.
- $P(y_t|\mathbf{y}_{<t})$ denotes the probability of the correct token given a set of previously predicted tokens

This leads to the optimization objective for a generic NLP problem which is:

$$\theta^* = \arg\min_{\theta} \left(-\sum_{t=1}^{T} \log P(y_t \mid \mathbf{y}_{<t}; \theta) \right)$$

where θ denotes all learnable weights in the model.

In addition to the ground truth for previous sequences, HMER's also require the actual image for processing. This means that the loss function in the HMER context can be defined as:

$$L = -\sum_{t=1}^{|Y|} \log P(y_t \mid \mathbf{y}_{<t}; \mathbf{I})$$

where

- $\mathbf{I}$ is a single image.
- $|Y|$ is the length of valid (non white space)tokens in the ground truth sequence, $\mathbf{y}$

This leads us to optimization objective for HMER which is

$$\theta^* = \arg\min_{\theta} \left(-\sum_{(\mathbf{I},\mathbf{Y})\in\mathcal{D}} \sum_{t=1}^{|Y|} \log P(y_t \mid \mathbf{y}_{<t}, \mathbf{I}; \theta) \right)$$

where $\mathcal{D}$ is the custom mathdataset loader and $\mathbf{Y}$ is the ground truth for the image. As a result of this, we can minimize the loss over multiple images.

It is also important to note that some normalization on this objective had to be done to ensure optimization stability. Some equations are inherently longer than others and this can result in the model misinterpreting some values during runtime. For context, say we have an equation $x + y$ (3 tokens) and let us say the model gets an error of 2.0 per token which implies that the total loss is 6.0. Again, say we have a 30-token equation but the model get's 0.5 per token so the total loss is 15.0. We can see that clearly the model is better at predicting the longer 30-token equation than the shorter one , but the model will think it's worse at it because it has a higher total loss. For this reason it is necessary to normalize this total loss.

$$\implies \theta^* = \arg\min_{\theta} \left(-\sum_{(\mathbf{I},\mathbf{Y})\in\mathcal{D}} \frac{1}{|Y|} \sum_{t=1}^{|Y|} \log P(y_t \mid \mathbf{y}_{<t}, \mathbf{I}; \theta) \right)$$

Additionally, at run time , because the summation will (ideally) shrink on

further epochs (complete pass of dataset through the model), it was necessary to normalize the final loss of each epoch for epoch loss comparison later on.

$$\implies \theta^* = \arg\min_{\theta} \left(-\frac{1}{|\mathcal{D}|} \sum_{(\mathbf{I},\mathbf{Y})\in\mathcal{D}} \frac{1}{|Y|} \sum_{t=1}^{|Y|} \log P(y_t \mid \mathbf{y}_{<t}, \mathbf{I}; \theta) \right)$$

.

Algorithm 6 and 7 denote the steps used for training. Before moving on, it is important to note that a concept called teacher forcing was used during training. When the model is trying to predict the next token, it uses the previous token it predicted at run time. Teacher forcing makes the model use the previous ground truth token instead for prediction during training. This is necessary because the model's weights for decoding are randomly initialized, so at the beginning of training using it's previous tokens to determine the next token will almost never yield good results. However, because we are "forcing" the correct history on it, it thinks it never makes mistakes and will not learn how to recover from them. To mitigate this, a decaying teacher forcing ratio was implemented such that at further epochs, the model uses teacher forcing less and less.

Finally, GRU's still suffer from the exploding gradient problem. In order to mitigate this gradient clipping was used. If the L_2 norm is greater than 5 then the gradient scaled down in such a way that the L_2 norm is 5. This method was specifically chosen because it still preserves the direction of the gradient vector.

2.3 Counting-Aware Network (CAN)

As mentioned before WAP made use of a coverage vector to specifically highlight pixels of the 2d feature map it has been to already which helped fix the lack of coverage problem. However, the issue with this is that, in cases where symbols are written close to each other while also being tightly spaced, if the attention mechanism has already been to that area, the coverage vector will tell the model to pay more attention to other areas. The reason this is a huge problem is because, the model can under-parse. So there is less of a chance

for the model to 1) go back to that area to check if it missed out on any symbols and 2) correct parsed symbols , if need be. Additionally, although this is not as frequent but the coverage vector can still lead to over-parsing, that is outputting more symbols than those that are present and the reason for this is because the coverage vector might make the attention mechanism linger on a specific character for longer than needed.

Bohan Li et al, proposed an extension to encoder-decoder based models (like WAP) to fix this problem. These are known as Counting Aware Networks and this will be the final implementation of the HMER for this project. CAN's are a network that jointly optimize two tasks namely, HMER and symbol counting. The idea behind this is to design a weakly-supervised counting module that can predict the number of each symbol class without the symbol-level position annotations, and then plug it into the WAP model in our case [8].

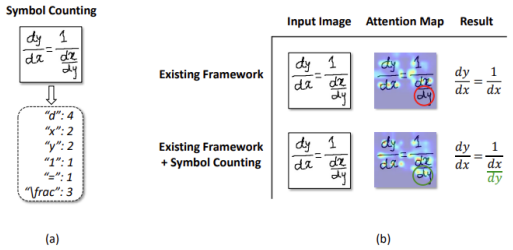

Figure 13: Typical Symbol Counting Task

Figure 13 shows the comparison and how the counting module improves the WAP module[8]. The counting module first see's that in the provided input image, there are 6 unique symbols. It then goes on to count how many of those unique symbols each , are present in the image. We can then see that on the right the WAP model missed 2 symbols, while the CAN model was able to capture those missed symbols.

2.3.1 The Counter Module

While the original CAN implementation utilizes a Multi-Scale Counting Module with spatial sum-pooling, this project uses a different, variant using Global

Average Pooling (GAP) for a good balance between performance and computational cheapness. This approach was heavily inspired by Zhou et al's proof that a GAP layer followed by a linear layer forces the channels of a CNN to detect objects [37].

Recall that each channel that the encoder outputs is looking for specific features i.e fraction bars or curves for 2's and 3's. Higher numbers denote where those features are located in the spatial grid, so for example, high values for the channel that detects fraction bars in the middle of the grid , mean that there is a fraction bar somewhere in the middle of the provided image. Since counting does not care about spatial relationships, it follows that we can just average those values and, for example, if the resulting value is high then it means that , the feature is present in the image. Note that even higher values mean that there could be more than one of these features present in the image. Following this, a linear connection is made to the output layer (which is of the size of the vocabulary) so that the model has the ability to learn the weighted combination of features that make up math symbols. The result yielded is a vector $\hat{\textbf{count}}$ of the predicted count of symbols.

With this vector alone it is not yet possible for the GRU to benefit from counts. We use a process called dynamic subtraction to dynamically reduce the number of counts in this vector at runtime to denote the number of current symbols left in image. This is done by creating a one hot vector that signifies the predicted token the model outputs and then subtracting it from the current count of tokens. It is important to note that in the event that the model predicts wrongly, this logic can lead to negative counts. As a result this subtraction is passed through the ReLU function to prevent negative counts. This remaining count vector is then concatenated with the previous gru inputs from before (in WAP), passed into the gru along with the previous hidden state to then generate the new hidden state.

2.3.2 Training the CAN model

Algorithm 8: CAN Single Batch Training Step

Input: Images $\mathbf{I}$, Targets $\mathbf{Y}$ of length T, Encoder E, WSCM W, Decoder D, Optimizer, CE Loss $\mathcal{L}_{seq}$, MSE Loss $\mathcal{L}_{count}$, TF Ratio ρ

$Optimizer$.zero_grad(); $Loss_{seq} \leftarrow 0$; $valid\_steps \leftarrow 0$

// Phase 1 & 2: Feature Extraction, Initial Counts, and Ground Truth

$\mathbf{A} \leftarrow E(\mathbf{I})$ // Extract spatial feature map

$\hat{\mathbf{v}} \leftarrow W(\mathbf{A})$ // Predict initial symbol counts $[B, V]$

$\mathbf{v} \leftarrow$ Zero matrix $[B, V]$

for ***each*** $Y_i \in \mathbf{Y}$ **do**

  for ***each*** $token \in Y_i \setminus \{SOS, EOS, PAD, UNK\}$ **do**

    $\mathbf{v}[i, token] \leftarrow \mathbf{v}[i, token] + 1$

  end

end

$Loss_{count} \leftarrow \mathcal{L}_{count}(\hat{\mathbf{v}}, \mathbf{v})$

// Phase 3: Decoder Initialization

$\mathbf{s}_0, \mathbf{c}_0 \leftarrow \mathbf{0}$ // Initialize hidden state and coverage

$\mathbf{c}_1^{dyn} \leftarrow \hat{\mathbf{v}}$ // Clone initial counts for dynamic subtraction

// Phase 4: Sequence Decoding Loop

for $t = 1 \dots T-1$ **do**

  $r \sim U(0, 1)$

  if $t = 1$ ***or*** $r < \rho$ **then**

    $y_{in} \leftarrow \mathbf{Y}_{t-1}$, $y_{chosen} \leftarrow \mathbf{Y}_t$ // Teacher Forcing

  else

    $y_{in} \leftarrow \arg\max(\hat{\mathbf{y}}_{t-1})$, $y_{chosen} \leftarrow \arg\max(\hat{\mathbf{y}}_t)$ // Use own predictions

  end

  $\hat{\mathbf{y}}_t, \mathbf{s}_t, \alpha_t, \mathbf{c}_t \leftarrow D(y_{in}, \mathbf{s}_{t-1}, \mathbf{A}, \mathbf{c}_{t-1}, \mathbf{c}_t^{dyn})$

  // Dynamic Count Subtraction

  $\mathbf{m} \leftarrow \text{OneHot}(y_{chosen}, V)$

  $\mathbf{c}_{t+1}^{dyn} \leftarrow \text{ReLU}(\mathbf{c}_t^{dyn} - \mathbf{m})$ **if** y_{chosen} is valid math **else** $\mathbf{c}_t^{dyn}$

  // Sequence Loss Accumulation

  if $\mathbf{Y}_t \neq PAD\_IDX$ **then**

    $Loss_{seq} \leftarrow Loss_{seq} + \mathcal{L}_{seq}(\hat{\mathbf{y}}_t, \mathbf{Y}_t)$; $valid\_steps \leftarrow valid\_steps + 1$

  end

end

// Phase 5: Joint Optimization

$Loss_{total} \leftarrow (\frac{Loss_{seq}}{valid\_steps}) + (\lambda \times Loss_{count})$

$Loss_{total}$.backward()

ClipGradients(E, W, D, max_norm $= 5.0$); $Optimizer$.step()

return $Loss_{total}, (Loss_{seq}/valid\_steps), Loss_{count}$

Algorithm 9: CAN Model Training Loop

Input: Dataset Loader $\mathcal{D}$, Encoder E, WSCM W, Decoder D, Total Epochs N

`// Initialization and Transfer Learning`

Initialize Adam Optimizer with learning rates η_E, η_D

Initialize StepLR Scheduler ($\gamma = 0.5$, step=4)

if *WAP Baseline Checkpoint exists* **then**

  Load E parameters entirely from baseline

  Load D parameters excluding GRU sequence weights

  Initialize W and D_{GRU} randomly

end

for $epoch = 1 \dots N$ **do**

  E.train(), W.train(), D.train()

  $EpochLoss_{total}, EpochLoss_{seq}, EpochLoss_{count} \leftarrow 0$

  $\rho \leftarrow \max(0.2, 1.0 - (epoch \times 0.1))$ `// Decay Teacher Forcing`

  for ***each*** *batch* $(\mathbf{I}, \mathbf{Y}) \in \mathcal{D}$ **do**

    $L_{tot}, L_{seq}, L_{cnt} \leftarrow$

    TrainStep$(\mathbf{I}, \mathbf{Y}, E, W, D, Optimizer, \mathcal{L}_{seq}, \mathcal{L}_{count}, \rho)$

    $EpochLoss_{total} \leftarrow EpochLoss_{total} + L_{tot}$

    $EpochLoss_{seq} \leftarrow EpochLoss_{seq} + L_{seq}$

    $EpochLoss_{count} \leftarrow EpochLoss_{count} + L_{cnt}$

  end

  `//` $|\mathcal{D}|$ `normalization`

  $\overline{EpochLoss} \leftarrow EpochLoss/|\mathcal{D}|$

  SaveCheckpoint$(epoch, E, W, D, Optimizer, \overline{EpochLoss})$

  $Scheduler$.step()

end

As we want to get the deviation between true counts and predicted counts, the counter's loss function can be defined using MSE, MAE e.t.c. MSE was chosen in this case for it's sensitivity to larger deviations from the ground truth.

$$L_{count} = \frac{1}{|V|} \sum_{i=1}^{|V|} (\hat{\mathbf{v}}_i(\mathbf{I}) - \mathbf{v}_i(\mathbf{I}))^2$$

where V is the size of the vocabulary.

Because the counter module now influences the predictions, it will be necessary to add it's loss to the log loss from before. The loss functon for CAN is formally denoted as,

$$Loss_{total} = Loss_{seq} + \lambda Loss_{count}$$

where λ denotes L_{count} weighting. So we can redefine our objective function to be

$$\theta^* = \arg\min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{I},\mathbf{Y})\in\mathcal{D}} \left[-\frac{1}{|Y|} \sum_{t=1}^{|Y|} \log P(y_t \mid \mathbf{y}_{<t}, \mathbf{I}; \theta) + \lambda \frac{1}{|V|} \sum_{i=1}^{|V|} (\hat{\mathbf{v}}_i(\mathbf{I};\theta) - \mathbf{v}_i(\mathbf{I}))^2 \right]$$

Algorithms 8 and 9 detail the steps in training. Most of it is very similar to the WAP training pipeline. However, except for the new additions talked about before, a scheduler was added into the training loop to reduce learning rate at later epochs. More on this will be talked about in the evaluation section.

2.4 Math Engine

Sympy was used to create the Math Engine. As a result of using the normalized label tag from the MathWritingDataset, math functions like \cos, \sin, are not present in the vocabulary. Instead the model has to read each individual letter to form these functions. As a result, because sympy requires these actual latex tokens, some sanitization on all allowed functions like these was performed. For example , if the an image has $\cos(x)$, the model will actually output $c\ o\ s\ (\ x\ )$. So regex rules were used to find leading patterns like this for all the functions and replace them with something like $\backslash\cos(x)$ for the math engine to solve properly.

Algorithm 10: Symbolic Mathematical Evaluation and Routing

Input: Parsed symbolic object $\mathcal{O}$
Output: Evaluated mathematical result or Error string $\mathcal{R}$

```
// Phase 1: Input Validation and Sanitization
if O is None then
    return Error: Invalid parsing
end
if O is of type List then
    if |O| > 0 then
        O ← O[0]                                  // Extract primary expression
    else
        return Error: Empty input
    end
end
if type(O) ∈ {Integer, Float} then
    O ← Sympify(O)                                // Cast to symbolic type
end

// Phase 2: Calculus and Advanced Operations
if O contains {Integral, Derivative, Limit, Sum, Product} then
    R ← Attempt Evaluation(O)
    if Evaluation is successful then
        if FreeSymbols(R) = ∅ then
            return Simplify(R)                    // Collapse pure numbers
        end
        return R
    else
        return Error: Calculus evaluation failed
    end
end

// Phase 3: Basic Arithmetic Routing
if O is not an Equation then
    if FreeSymbols(O) = ∅ then
        return Simplify(O)                        // e.g., forces '2 + 2' to '4'
    end
end

// Phase 4: Equation Formatting and Algebraic Solver
if O is not an Equation then
    O ← Equation(O, 0)                            // Format implicitly as O = 0
end

R ← Attempt Algebraic Solve(O)
if Solver is successful then
    return R
else
    return Error: Algebraic solver failed
end
```

Algorithm 10 details the routing algorithm for solving equations. As a reminder, the solver is to be allowed to solve algebraic, transcendental, calculus based and basic arithmetic problems. Therefore,

- Phase 1 involves extracting the primary expression itself. If the input is a simple number instead, it just converts that number into a sympy object.
- Phase 2 checks to see if the sympy object contains specific symbols like integrals or derivatives e.t.c. Additionally it goes on to check if there are any variables left in it's evalution (free symbols) before outputting the result. The user might not necessarilly want to submit a definite integral for instance. This phase solves that.
- Phase 3 checks to see if the user wants to evaluate basic arithmetic. For this to be the case, the sympy object must not be an equation and must not have any variables.
- Finally, phase 4 is for actual equation routing (algebraic and transcendental). In the event the user does not write an equation with an $=$ it is assumed that they want their expression evaluated with $= 0$ i.e $x^2 + 2x + 5 = 0$. The engine goes on to then attempt to solve the resulting equation.

It is very important to note that this function expects a sympy object as input. As a result, when being, used, the parsed latex is first sanitized by the custom sanitizer and then uses sympy's built in "latex2sympy" function for this conversion[32].

2.5 The WebApp

The webapp for this was made using gradio. Gradio is an open-source python librabry that allows developers to create websites with ease[23] for machine learning projects. The major talking point about the webapp comes from the fact that input images from users will be of different sizes and possibly different colours, additionally, because the model specifically takes images of a specific size, some pre-processing on input images has to be carried out for these cases.

To account for the first case, the Otsu's algorithm was used alongside a binary flag. The aim of Otsu's algorithm is to determine the optimal threshold value that separates two colours from each other. For example with blue text on a red background, Otsu's algorithm finds a middle ground between these two colours. Once Otsu finds this value, a binary flag is executed. Any pixel darker than this threshold becomes pure black and vice versa for lighter. To account for the second case, this image then goes through the same pre-processing rasterization process in algorithm 5 to match the required dimensions[17].

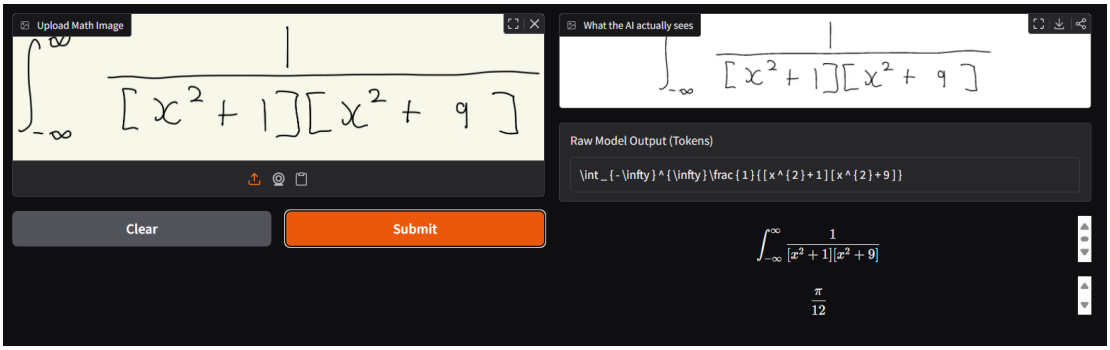

Figure 14: Web-App UI

Figure 14 shows the webapp UI with a sample image passed through. This image is preprocessed using the previous preprocess logic spoken about in the last paragraph. The result is then passed through to the HMER shown in the top right section of the figure. Below that shows the raw tokens the model outputs, following this is the sanitized input which is then passed to the MathEngine for solving. The resulting answer for the proposed expression on the input image is then displayed below the sanitzed LaTeX text.

It is important to note that the MathEngine can often come across intractable evaluations when parsing complex or malformed expressions. This can make the server thread hang indefinitely. To mitigate this, the MathEngine was wrapped in a multi-processed timeout handler, which kills the process and outputs just the read equation and an error text so the user knows what it going on at all times. This is shown in figure 15

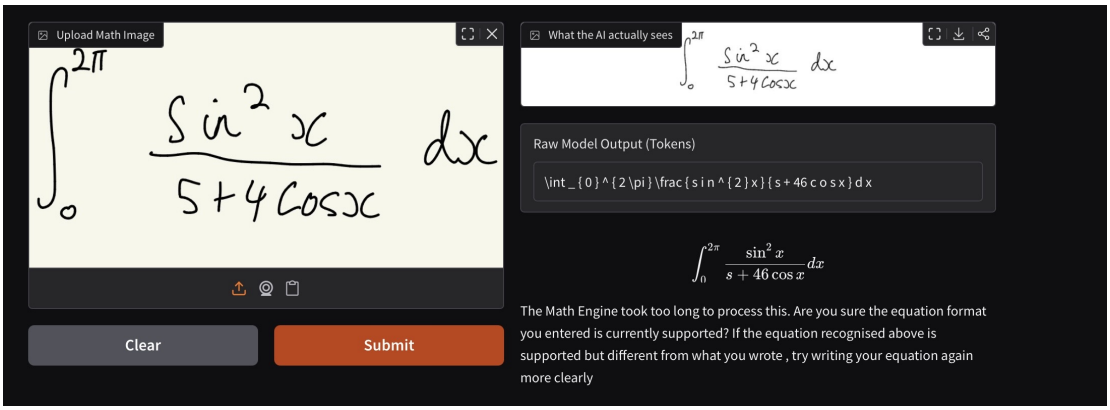

Figure 15: Malformed Equation Handling

3 Evaluation

3.1 WAP results

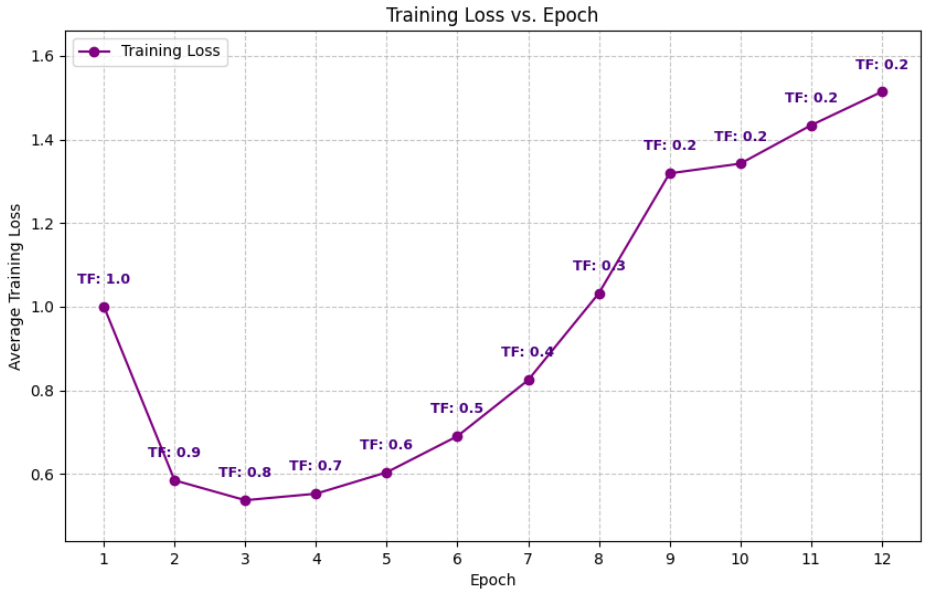

Figure 16: WAP training loss curve

The number of epochs N was set to 12. This was a good compromise over regular training methods because , for reference, one epoch processed over 400,000 images. This means that by the time the 12th epoch was finished, the model would have seen about 5,000,000 images in total. This allows the model

to make enough gradient updates to at least converge towards a local minimum. Figure 16, which shows the loss curve during training, shows that the model was learning for the first few epochs. Notice also that the teacher forcing ratio is decaying as the epoch number increases, so it is expected that the loss would increase again. However, the loss never dropped back down even after the teacher forcing ratio stayed the same. Infact, the initial number of epochs was set to 10 but because this continual increase in loss was observed, it was necessary to test a few more epochs (11 and 12).

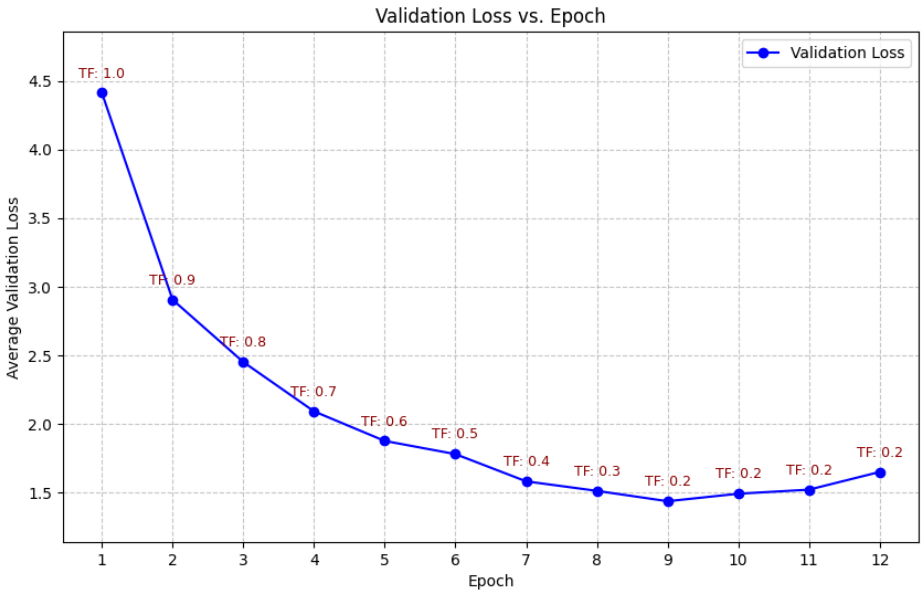

Figure 17: WAP validation loss curve

The WAP validation loss curve is shown above. This shows that the model did learn , but we also notice that the same rise in loss starts to happen again after epoch 9. This implied learning instability and the culprit for this was found to be the learning rates. For initial epochs, they were sufficient but for further epochs, it made the model overshoot past the local minimum it was converging towards. Instead of training again from scratch, epoch 9 was picked and used as the baseline for the CAN implementation as it had the lowest validation loss.

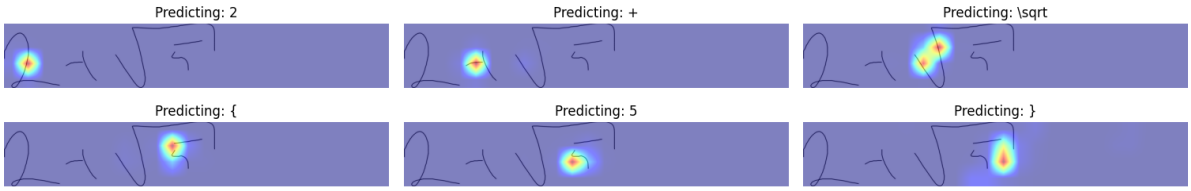

Figure 18: WAP Attention map

In addition to this, since we have access to the attention scores $\alpha_{t,i}$, during inference, these were also stored so we can track and see where exactly the model looks at at each time step and it's predictions for those areas. Figure 18 shows this very well. We can see that the model scans the image from left to right, while also highlighting areas it pays attention to for each time step t. The model successfully predicted that the handwritten math expression was $2 + \sqrt{5}$ while also obeying the grammatical rules for a LaTeX sequence.

3.2 CAN results

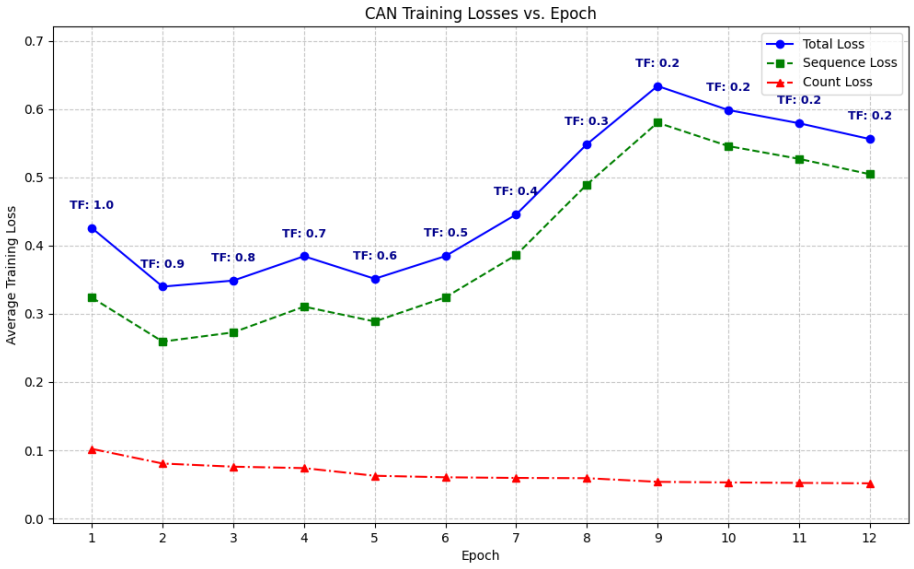

Figure 19: CAN training loss curve

Figure 19 shows the training loss curves for the CAN model. Note that λ was chosen to be 1 here in order to weight both the sequence and count loss

equally. It was noticed that the count loss was relatively small, so a smaller λ value will make this even smaller which is not ideal. Conversely a bigger λ value might emphasize the count loss too much. In figure 20 notice that a similar pattern emerges with the trend of loss up until epoch 9 again , but this time after this epoch, it starts to reduce which implies much more stable training. A scheduler with a step size of 4 and γ=0.5 to halve the learning rate after 4 epochs each time was used. As a result of the lower learning rate by epoch 9, the loss started to drop after this point.

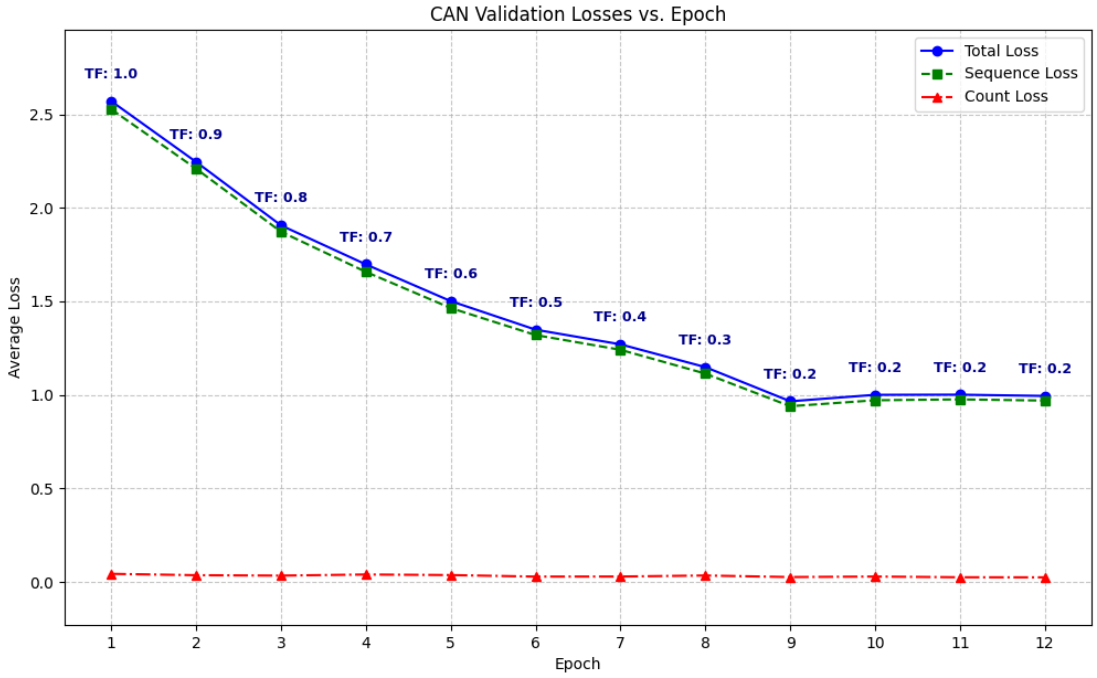

Figure 20: CAN validation loss curve

This figure shows the loss curve for validation. There is a steady drop in loss all the way up till epoch 9, afterwards, the loss rises very slightly and plateus around 1. This is likely due to slight overfitting to the training data on those epochs. As a result epoch 9 is chosen for the final model because it yields the lowest validation loss.

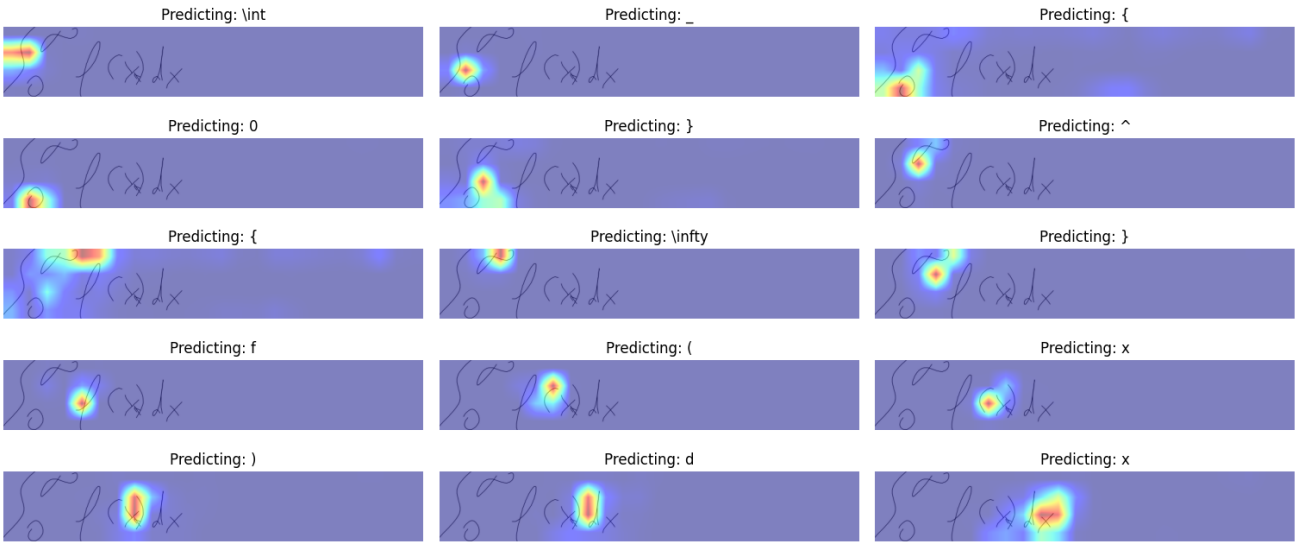

Figure 21: CAN Attention map

Lastly we observe the attention map above for the CAN model in figure 21. Notice that at the bottom, the count vector is shown in the format "symbol, count". The counter , for the most part, correctly counts the number of symbols in the image as well as intermediate tokens for LaTeX syntax correctness.

3.3 Expression Recognition rate

Expression recognition rate (expRate), defined as the percentage of correctly recognized expressions, is used to evaluate the performance of different methods on mathematical expression recognition . In addition to this, because expressions can be very lengthy, another standard is to define a $\leq n, n \in \mathbb{N}$ rate for tolerance of error. This means, the expression rate is tolerable to at most n token-level errors.[8] When gauging the models performance, this concept will be used was used while varying n. Formally speaking, the expression recognition rate formula is defined as,

$$\text{expRate} = \frac{\text{Number of exact matches}}{\text{Total number of tested expressions}}$$

With $n = 1$, the formula becomes

$$\text{expRate}(\leq 1) = \frac{\text{Number of matches with at most one symbol mismatch}}{\text{Total number of tested expressions}}$$

To implement this, we use the leventshtein distance between two sequences namely the the ground truth and predicted token sequence, which measures the number of edits required to turn the prediction into the exact target[25].

Table 2: expRate Metric on Test set

Evaluation Metric	**WAP Baseline**	**CAN**
Average Levenshtein Distance ↓	2.68	**1.72**
Exact Match (expRate, $n = 0$) ↑	29.97%	**45.75%**
expRate ($n \leq 1$) ↑	49.69%	**66.22%**
expRate ($n \leq 2$) ↑	63.03%	**76.40%**
expRate ($n \leq 3$) ↑	72.59%	**83.65%**

Table 2 details the results for the best version of both models. As we can see the CAN model is superior in every aspect wrt the WAP baseline. We also notice that expRate(n=0) is an extremely strict metric as the model was not even able to make it past 50%. However as the tolerance is varied , we notice that the model is actually predicting correct tokens for the most part but making some mis-predictions down the line. This however falls in line with the expected behavior of the CAN model which implies no fundamental flaws in the implementation[8]

4 Conclusion and Further Work

A working and robust handwritten equation solver was made. The HMER shows very good results for the most part but in some cases fails to tell the difference between some very similar symbols (e.g s and 5), further overparsing still seems to occur in some cases , for example $\frac{d}{dx}\left(\frac{e^{-2x}\cos(\pi x)}{x^2+1}\right)$, because) of the cos and the big right bracket are close, the model predicted an extra), when viewing the receptive field around the cos. In addition, the Math Engine has proven to be robust enough to solve the intended types of equations

The HMER itself already shows good results from training over 9 epochs

only, but started overfitting slightly afterwards. To prevent this, a future system could make use of regularization techniques such as L_2 regularization on L_{count} for example. Further, the MathEngines solving capabilities are limited. A future system could allow for some extra more complex features such as solving equations that require the use of the Lambert W function (e.g $xe^x = 3$). Additonally , extra preprocessing could be implemented in a future system to allow for HME from cameras to solved.

5 Acknowledgements

I would like to start by thanking my Project Supervisor Weiren Yu for assisting me throughout this project. Specifically, my core intuition on Artificial Neural Networks and Backpropagation was inspired by him. This made it much easier to understand the formulation of the WAP loss function and how the model as a whole tries to minimize it's loss. Alongside my project supervisor Weiren Yu, I would like to thank my Second Assessor Dr Subhash Lakshminarayana for both taking the time out to arrange meetings with me in order to give constructive feedback, specifically on my progress report. This helped in piecing together what needs to be included in the final report.

References

[1] Francisco Alvaro, Joan-Andreu Sánchez, and José-Miguel Benedí. Recognizing printed and handwritten mathematical expressions using 2d stochastic context-free grammars and hidden markov models. *Information Sciences*, 2014.

[2] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 1994.

[3] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning

phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[4] Noam Chomsky. On certain formal properties of grammars. *Information and control*, 1959.

[5] P.A. Chou. Recognition of equations using a two -dimensional stochastic context -free grammar, 1989.

[6] Jack D. Cowan. Mcculloch-pitts and related neural nets from 1943 to 1989. https://www.sciencedirect.com/science/article/pii/S0092824005800059, 2005.

[7] A. Kadar et al. Tutorial #15: Parsing i context-free grammars and the cyk algorithm. https://rbcborealis.com/research-blogs/tutorial-15-parsing-i-context-free-grammars-and-cyk-algorithm/, 2021.

[8] Bohan Li et al. When counting meets hmer: Counting-aware network for handwritten mathematical expression recognition. https://arxiv.org/pdf/2207.11463, 2022.

[9] Diego Andina et al. Neural networks historical review. https://www.researchgate.net/publication/285325023_Neural_Networks_Historical_Review, 2007.

[10] Erik G. Miller et al. Ambiguity and constraint in mathematical expression recognition, 1998.

[11] Gustavo Almeida et al. Arabica coffee samples classification using a multilayer perceptron neural network. https://www.researchgate.net/publication/330704998_Arabica_coffee_samples_classification_using_a_Multilayer_Perceptron_neural_network, 2018.

[12] Jianshu Zhang et al. Watch, attend and parse: An end-to-end neural network based approach to handwritten mathematical expression recognition. http://home.ustc.edu.cn/~xysszjs/paper/PR2017.pdf, 2017.

[13] Kuruva Satya Ganesh. What's the role of weights and bias in a neural network? https://medium.com/data-science/

whats-the-role-of-weights-and-bias-in-a-neural-network-4cf7e9888a0f, 2020.

[14] Philippe Gervais et al. Mathwriting: A dataset for handwritten mathematical expression recognition. https://arxiv.org/html/2404.10690v1, 2024.

[15] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 2010.

[16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[17] Bahaa El-Din Helmy. Understanding otsu's method for image segmentation. https://www.baeldung.com/cs/otsu-segmentation, 2025.

[18] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017.

[19] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015.

[20] Sebastien Jean, Kyunghyun Cho, Roland Memisevic, and Yoshua Bengio. On using very large target vocabulary for neural machine translation. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2015.

[21] Tadao Kasami. An efficient recognition and syntax-analysis algorithm for context-free languages. Technical report, Air Force Cambridge Research Lab, 1965.

[22] Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. Dying ReLU and initialization: Theory and numerical examples. *Communications in Computational Physics*, 2020.

[23] Hector Martinez. Introduction to gradio for building interactive applications. https://pyimagesearch.com/2025/02/03/

introduction-to-gradio-for-building-interactive-applications/, 2025.

[24] Marvin Minsky and Seymour A Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[25] Shaoni Mukherjee. Levenshtein distance: A comprehensive guide. https://www.digitalocean.com/community/tutorials/levenshtein-distance-python, 2025.

[26] Joshua Noble. What is an encoder-decoder model. https://www.ibm.com/think/topics/encoder-decoder-model, 2025.

[27] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Cornell Aeronautical Laboratory*, 1958.

[28] R.Rojas. *Neural Networks: A Systematic Introduction*. Springer-Verlag, 1996.

[29] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[30] StatPearls Publishing. Anatomy of neurons. https://www.ncbi.nlm.nih.gov/books/NBK526047/figure/article-29802.image.f1/, 2023.

[31] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, 2014.

[32] SymPy. Sympy 1.14.0 documentation. https://docs.sympy.org/latest/index.html, 2026.

[33] Ernesto Tapia and Raúl Rojas. Recognition of on-line handwritten mathematical expressions using a minimum spanning tree construction and symbol dominance. In *International Conference on Graphics Recognition*. Springer, 2003.

[34] Nguyen Van et al. A short review for handwritten math expression recognition. https://www.sciencedirect.com/science/article/pii/S1877050924007014, 2024.

[35] Daniel H Younger. Recognition and parsing of context-free languages in time n^3. *Information and control*, 1967.

[36] Jiawei Zhang. Basic neural units of the brain: Neurons, synapses and action potential. `https://arxiv.org/pdf/1906.01703`, 2019.

[37] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. Learning deep features for discriminative localization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.